

7/7 pruebas pasando. Ahora empaqueto el proyecto completo:---

Resumen de entregables

Archivos creados (13 archivos de código)

```
el_ultimo_bastion/
├── main.py                ← Punto de entrada
├── requirements.txt
├── src/
│   ├── core/
│   │   ├── settings.py    ← Constantes globales
│   │   ├── event_bus.py   ← Observer / bus de eventos
│   │   ├── asset_loader.py ← Carga y caché de assets
│   │   ├── state_machine.py ← Gestor de pantallas
│   │   └── game.py        ← Game loop principal ★
│   ├── screens/
│   │   ├── base_screen.py ← Interfaz abstracta
│   │   ├── main_menu.py   ← Menú principal
│   │   ├── gameplay.py    ← Pantalla de juego
│   │   ├── pause_screen.py ← Pausa
│   │   └── defeat_screen.py ← Game Over
│   └── ui/
│       └── button.py       ← Componente botón reutilizable
├── tests/
│   ├── test_event_bus.py   ← 7 tests ✅ pasando
│   └── test_state_machine.py
```

Explicación técnica de cada módulo clave

settings.py — fuente única de verdad para resolución (1280×720), FPS objetivo (60), paleta de colores medieval y nombres de estados. Cualquier cambio de configuración ocurre aquí y se propaga a todo el proyecto.

event_bus.py — implementa el patrón Observer con un diccionario **evento** → **[callbacks]**. Es singleton para que cualquier módulo lo obtenga sin pasarlo como parámetro. Los tests de **test_event_bus.py** verifican suscripción, publicación, desuscripción, duplicados y **clear()**, todos en verde sin necesitar Pygame.

asset_loader.py — centraliza la carga de imágenes, fuentes y sonidos con caché de diccionario. Si un asset no existe, devuelve un placeholder magenta en lugar de crashear, lo que es crítico durante el desarrollo cuando los artistas aún no han entregado los sprites.

state_machine.py — gestiona las transiciones de pantalla con un mecanismo de "transición pendiente": cuando se llama `change()`, el cambio real se aplica al inicio del siguiente frame, evitando modificar el estado activo a mitad de su propio `handle_events()`.

game.py — orquesta todo. Inicializa Pygame con `DOUBLEBUF` para renderizado sin tearing, aplica el tope `MAX_DELTA_TIME = 0.05s` al delta time para evitar saltos de posición por pausas del sistema operativo, y llama al loop en el orden correcto: eventos → update → clear → draw → flip.

base_screen.py — contrato abstracto (ABC) que todas las pantallas deben cumplir. Garantiza que la `StateMachine` pueda tratar cualquier pantalla de forma uniforme sin saber su tipo concreto.

button.py — componente UI con tres estados visuales (normal, hover, presionado) usando la paleta medieval. Maneja sus propios eventos internamente; la pantalla solo le pasa cada evento con `button.handle_event(event)`.

Instrucciones de ejecución

1. Instalar dependencia
`pip install pygame>=2.5.0`

2. Ejecutar el juego (desde la raíz del proyecto)
`python main.py`

3. Ejecutar las pruebas unitarias (no requieren Pygame)
`python -m unittest tests.test_event_bus -v`

Controles disponibles en esta iteración:

Tecla / Acción	Resultado
Clic en "Jugar"	Entra a la pantalla de gameplay
Clic en "Salir"	Cierra el programa limpiamente
ESC en gameplay	Abre la pantalla de pausa
ESC en pausa	Reanuda el juego
"Menú Principal"	Regresa al menú desde pausa
Alt+F4	Cierre estándar (Windows)

Cerrar ventana Cierre limpio con
`pygame.quit()`

El siguiente sprint natural es integrar el `MapSystem` con un grid real y el `WaveSystem` con spawning de enemigos placeholder, que ya tienen sus carpetas y `__init__.py` listos para recibir código.

Codigo de los Archivos

```
"""
src/core/settings.py
=====
Constantes globales y configuración de runtime del videojuego.
Centraliza todos los valores de configuración para facilitar
ajustes sin modificar otros módulos.
"""

# -----
# Ventana y pantalla
# -----
SCREEN_WIDTH: int = 1280
SCREEN_HEIGHT: int = 720
SCREEN_TITLE: str = "El Último Bastión"
TARGET_FPS: int = 60

# -----
# Colores (R, G, B)
# -----
class Colors:
    BLACK    = (0, 0, 0)
    WHITE    = (255, 255, 255)
    DARK_GRAY = (30, 30, 30)
    GRAY     = (80, 80, 80)
    LIGHT_GRAY = (160, 160, 160)

    # Paleta temática Fantasía Medieval Oscura
    DARK_STONE = (40, 35, 45) # Fondo principal
    DEEP_PURPLE = (55, 25, 75) # Acento oscuro
    BLOOD_RED = (140, 20, 20) # Peligro / enemigos
    GOLD = (212, 175, 55) # Recurso oro
    MANA_BLUE = (50, 120, 220) # Recurso maná
    FOREST_GREEN = (34, 85, 34) # Elementos naturales
    PARCHMENT = (220, 200, 160) # Texto principal UI

# -----
# Paths de assets (relativos a la raíz del proyecto)
# -----
```

```

class Paths:
    ASSETS    = "assets"
    SPRITES   = "assets/sprites"
    SOUNDS    = "assets/sounds"
    FONTS     = "assets/fonts"
    DATA     = "data"
    MAPS      = "data/maps"

# -----
# Parámetros del Game Loop
# -----
MAX_DELTA_TIME: float = 0.05 # Tope de dt en segundos (evita saltos por pausas del
SO)

# -----
# Identificadores de estados (pantallas)
# -----
class States:
    MAIN_MENU    = "main_menu"
    GAMEPLAY     = "gameplay"
    PAUSE        = "pause"
    DEFEAT       = "defeat"

```

"""

src/core/event_bus.py

=====

Implementación del patrón Observer como bus de eventos global.

Permite que los sistemas publiquen y escuchen eventos sin conocerse entre sí, garantizando bajo acoplamiento en toda la arquitectura.

Uso:

```

bus = EventBus.get_instance()
bus.subscribe("enemy_died", mi_callback)
bus.publish("enemy_died", {"gold": 10})

```

"""

```

from __future__ import annotations
from collections import defaultdict
from typing import Any, Callable

```

```

class EventBus:

```

"""

Bus de eventos singleton.

Attributes:

```
    _instance: Instancia única (patrón Singleton).
    _listeners: Diccionario que mapea nombre_evento → lista de callbacks.
    """
```

```
_instance: EventBus | None = None
```

```
def __init__(self) -> None:
    # Diccionario: evento → lista de funciones suscritas
    self._listeners: dict[str, list[Callable]] = defaultdict(list)
```

```
# -----
# Singleton
# -----
```

```
@classmethod
def get_instance(cls) -> "EventBus":
    """Devuelve la instancia única del bus. La crea si no existe."""
    if cls._instance is None:
        cls._instance = cls()
    return cls._instance
```

```
@classmethod
def reset(cls) -> None:
    """Destruye la instancia actual. Útil para tests."""
    cls._instance = None
```

```
# -----
# API pública
# -----
```

```
def subscribe(self, event_name: str, callback: Callable) -> None:
    """
    Suscribe un callback a un evento.
```

```
    Args:
        event_name: Nombre del evento (p.ej. "enemy_died").
        callback: Función que recibe los datos del evento.
    """
```

```
    if callback not in self._listeners[event_name]:
        self._listeners[event_name].append(callback)
```

```
def unsubscribe(self, event_name: str, callback: Callable) -> None:
    """
    Elimina la suscripción de un callback.
```

```
    Args:
        event_name: Nombre del evento.
        callback: Función a desuscribir.
```

```

"""
    if callback in self._listeners[event_name]:
        self._listeners[event_name].remove(callback)

def publish(self, event_name: str, data: Any = None) -> None:
    """
        Publica un evento, notificando a todos los suscriptores.

    Args:
        event_name: Nombre del evento a publicar.
        data: Datos opcionales que se pasan al callback.
    """
    for callback in list(self._listeners[event_name]):
        callback(data)

def clear(self) -> None:
    """Elimina todas las suscripciones. Útil al cambiar de pantalla."""
    self._listeners.clear()

"""
src/core/asset_loader.py
=====
Carga y caché centralizada de todos los recursos del juego
(sprites, fuentes, sonidos).

Principio: los assets se cargan UNA sola vez en memoria y se
reutilizan mediante un diccionario de caché, evitando lecturas
de disco repetidas durante el game loop.
"""

from __future__ import annotations
import os
import logging
import pygame

from src.core.settings import Paths

logger = logging.getLogger(__name__)

class AssetLoader:
    """
        Cargador de assets con caché interna.

        Todos los métodos de carga son estáticos con caché de clase,
        de modo que múltiples sistemas pueden llamarlos sin duplicar
        objetos en memoria.
    """

```

```

_images: dict[str, pygame.Surface] = {}
_fonts: dict[tuple, pygame.font.Font] = {}
_sounds: dict[str, pygame.mixer.Sound] = {}

# -----
# Imágenes
# -----

@classmethod
def get_image(cls, relative_path: str, convert_alpha: bool = True) -> pygame.Surface:
    """
    Carga una imagen desde disco o la devuelve desde caché.

    Args:
        relative_path: Ruta relativa a la raíz del proyecto.
        convert_alpha: Si True usa convert_alpha() para sprites con transparencia.

    Returns:
        pygame.Surface lista para renderizado.
    """
    if relative_path not in cls._images:
        full_path = os.path.join(relative_path)
        try:
            surface = pygame.image.load(full_path)
            cls._images[relative_path] = (
                surface.convert_alpha() if convert_alpha else surface.convert()
            )
            logger.debug("Imagen cargada: %s", relative_path)
        except FileNotFoundError:
            logger.warning("Imagen no encontrada: %s — usando placeholder", relative_path)
            cls._images[relative_path] = cls._create_placeholder(32, 32)

    return cls._images[relative_path]

@staticmethod
def _create_placeholder(width: int, height: int) -> pygame.Surface:
    """Crea una surface magenta como placeholder visual de asset faltante."""
    surface = pygame.Surface((width, height), pygame.SRCALPHA)
    surface.fill((255, 0, 255, 180))
    return surface

# -----
# Fuentes
# -----

@classmethod
def get_font(cls, name: str | None, size: int) -> pygame.font.Font:

```

```
"""
```

Carga o recupera una fuente del caché.

Args:

name: Ruta al archivo .ttf o None para la fuente del sistema.

size: Tamaño en puntos.

Returns:

pygame.font.Font lista para render de texto.

```
"""
```

```
key = (name, size)
```

```
if key not in cls._fonts:
```

```
    try:
```

```
        if name is None:
```

```
            cls._fonts[key] = pygame.font.SysFont("Arial", size)
```

```
        else:
```

```
            cls._fonts[key] = pygame.font.Font(name, size)
```

```
            logger.debug("Fuente cargada: %s @ %dpix", name or "SysFont", size)
```

```
    except Exception as exc:
```

```
        logger.warning("Error cargando fuente %s: %s — usando fallback", name, exc)
```

```
        cls._fonts[key] = pygame.font.SysFont("Arial", size)
```

```
    return cls._fonts[key]
```

```
# -----
```

```
# Sonidos
```

```
# -----
```

```
@classmethod
```

```
def get_sound(cls, relative_path: str) -> pygame.mixer.Sound | None:
```

```
    """
```

Carga o recupera un efecto de sonido del caché.

Returns:

pygame.mixer.Sound o None si el archivo no existe.

```
    """
```

```
    if relative_path not in cls._sounds:
```

```
        try:
```

```
            cls._sounds[relative_path] = pygame.mixer.Sound(relative_path)
```

```
            logger.debug("Sonido cargado: %s", relative_path)
```

```
        except Exception as exc:
```

```
            logger.warning("Sonido no encontrado: %s — %s", relative_path, exc)
```

```
            cls._sounds[relative_path] = None
```

```
    return cls._sounds[relative_path]
```

```
# -----
```

```
# Limpieza
```



```
# -----
```

```
@classmethod
```

```
def clear_cache(cls) -> None:
```

```
    """Libera todos los assets en memoria. Útil entre niveles."""
```

```
    cls._images.clear()
```

```
    cls._fonts.clear()
```

```
    cls._sounds.clear()
```

```
    logger.info("Caché de assets liberada.")
```

```
"""
```

```
src/core/state_machine.py
```

```
=====
```

Máquina de estados finitos que gestiona las pantallas del juego.

Cada estado (pantalla) es un objeto que implementa la interfaz:

- handle_events(events)
- update(dt)
- draw(screen)

La StateMachine delega cada fase del game loop al estado activo, y permite transiciones limpias entre pantallas.

```
"""
```

```
from __future__ import annotations
```

```
import logging
```

```
from typing import TYPE_CHECKING
```

```
import pygame
```

```
if TYPE_CHECKING:
```

```
    from src.screens.base_screen import BaseScreen
```

```
logger = logging.getLogger(__name__)
```

```
class StateMachine:
```

```
    """
```

Gestiona el estado activo del juego y las transiciones entre pantallas.

Attributes:

 _states: Registro de estados disponibles {nombre: instancia}.

 _current: Estado actualmente activo.

 _next_state: Estado al que se transitará en el próximo frame.

```
    """
```

```
def __init__(self) -> None:
```

```
    self._states: dict[str, "BaseScreen"] = {}
```

```

self._current: "BaseScreen | None" = None
self._next_state: str | None = None

# -----
# Registro de estados
# -----

def register(self, name: str, screen: "BaseScreen") -> None:
    """
    Registra una pantalla con un nombre identificador.

    Args:
        name: Clave del estado (usar constantes de Settings.States).
        screen: Instancia de la pantalla.
    """
    self._states[name] = screen
    logger.debug("Estado registrado: %s", name)

# -----
# Transiciones
# -----

def change(self, state_name: str) -> None:
    """
    Solicita una transición al estado indicado.
    La transición real ocurre al inicio del siguiente frame
    para evitar cambios de estado a mitad del loop.

    Args:
        state_name: Nombre del estado destino.
    """
    if state_name not in self._states:
        logger.error("Estado desconocido: %s", state_name)
        return
    self._next_state = state_name

def _apply_transition(self) -> None:
    """Aplica la transición pendiente si existe."""
    if self._next_state is None:
        return

    if self._current is not None:
        self._current.on_exit()

    self._current = self._states[self._next_state]
    self._current.on_enter()
    logger.info("Transición a estado: %s", self._next_state)
    self._next_state = None

```

```

# -----
# Delegación del game loop
# -----

def handle_events(self, events: list[pygame.event.Event]) -> None:
    self._apply_transition()
    if self._current:
        self._current.handle_events(events)

def update(self, dt: float) -> None:
    if self._current:
        self._current.update(dt)

def draw(self, screen: pygame.Surface) -> None:
    if self._current:
        self._current.draw(screen)

```

```

# -----
# Consultas
# -----

```

```

@property
def current_name(self) -> str | None:
    """Nombre del estado activo, o None si no hay ninguno."""
    for name, state in self._states.items():
        if state is self._current:
            return name
    return None

```

"""

src/screens/base_screen.py

=====

Clase base abstracta para todas las pantallas del juego.

Define la interfaz que la StateMachine espera de cualquier estado:

- on_enter() / on_exit(): ciclo de vida de la pantalla
- handle_events(events): procesamiento de inputs
- update(dt): lógica de actualización
- draw(screen): renderizado

"""

```

from __future__ import annotations
from abc import ABC, abstractmethod
from typing import TYPE_CHECKING

```

```

import pygame

```

```

if TYPE_CHECKING:
    from src.core.state_machine import StateMachine
    from src.core.event_bus import EventBus

class BaseScreen(ABC):
    """
    Interfaz abstracta que toda pantalla del juego debe implementar.

    Attributes:
        state_machine: Referencia para solicitar cambios de estado.
        event_bus: Bus de eventos para publicar/escuchar eventos globales.
    """

    def __init__(self, state_machine: "StateMachine", event_bus: "EventBus") -> None:
        self.state_machine = state_machine
        self.event_bus = event_bus

    # -----
    # Ciclo de vida
    # -----

    def on_enter(self) -> None:
        """
        Llamado cuando esta pantalla se vuelve activa.
        Sobreescibir para inicializar estado de la pantalla.
        """

    def on_exit(self) -> None:
        """
        Llamado justo antes de que esta pantalla se desactive.
        Sobreescibir para limpiar recursos o desuscribir eventos.
        """

    # -----
    # Game loop (abstractos — obligatorio implementar)
    # -----

    @abstractmethod
    def handle_events(self, events: list[pygame.event.Event]) -> None:
        """Procesa la lista de eventos del frame actual."""

    @abstractmethod
    def update(self, dt: float) -> None:
        """Actualiza la lógica de la pantalla. dt en segundos."""

    @abstractmethod
    def draw(self, screen: pygame.Surface) -> None:

```

```
"""Dibuja la pantalla sobre la surface proporcionada."""
```

```
"""
```

```
src/screens/main_menu.py
```

```
=====
```

```
Pantalla de Menú Principal.
```

```
Muestra el título del juego y dos opciones:
```

- Jugar → transición a Gameplay
- Salir → cierre del programa

```
"""
```

```
from __future__ import annotations
```

```
import pygame
```

```
from src.screens.base_screen import BaseScreen
```

```
from src.core.settings import SCREEN_WIDTH, SCREEN_HEIGHT, Colors, States
```

```
from src.core.asset_loader import AssetLoader
```

```
from src.ui.button import Button
```

```
class MainMenuScreen(BaseScreen):
```

```
    """
```

```
    Pantalla de menú principal.
```

```
    Layout:
```

- Fondo oscuro con efecto de degradado vertical simple.
- Título del juego centrado en la mitad superior.
- Subtítulo / tagline temático.
- Dos botones centrados verticalmente.

```
    """
```

```
    def on_enter(self) -> None:
```

```
        """Crea los botones cada vez que se entra al menú."""
```

```
        cx = SCREEN_WIDTH // 2
```

```
        btn_w, btn_h = 280, 56
```

```
        gap = 20
```

```
        # Posición Y del primer botón (centrado en la mitad inferior)
```

```
        start_y = SCREEN_HEIGHT // 2 + 40
```

```
        self._buttons = [
```

```
            Button(
```

```
                x=cx - btn_w // 2, y=start_y,
```

```
                width=btn_w, height=btn_h,
```

```
                label="Jugar",
```

```
                callback=self._on_play,
```

```

        font_size=26,
    ),
    Button(
        x=cx - btn_w // 2, y=start_y + btn_h + gap,
        width=btn_w, height=btn_h,
        label="Salir",
        callback=self._on_quit,
        font_size=26,
    ),
]

def on_exit(self) -> None:
    self._buttons = []

# -----
# Game loop
# -----

def handle_events(self, events: list[pygame.event.Event]) -> None:
    for event in events:
        for button in self._buttons:
            button.handle_event(event)

def update(self, dt: float) -> None:
    pass # El menú no tiene lógica dinámica en esta versión

def draw(self, screen: pygame.Surface) -> None:
    self._draw_background(screen)
    self._draw_title(screen)
    for button in self._buttons:
        button.draw(screen)

# -----
# Renderizado
# -----

def _draw_background(self, screen: pygame.Surface) -> None:
    """Dibuja un fondo con degradado vertical oscuro."""
    # Capa base (ya aplicada por Game._draw_background)
    # Añadimos un panel central semitransparente
    overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT),
pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 80))
    screen.blit(overlay, (0, 0))

    # Líneas decorativas horizontales (estética medieval)
    line_color = (Colors.DEEP_PURPLE[0], Colors.DEEP_PURPLE[1],
Colors.DEEP_PURPLE[2])

```

```

pygame.draw.line(screen, line_color, (80, SCREEN_HEIGHT // 2 - 10),
                  (SCREEN_WIDTH - 80, SCREEN_HEIGHT // 2 - 10), 1)
pygame.draw.line(screen, line_color, (80, SCREEN_HEIGHT // 2 - 6),
                  (SCREEN_WIDTH - 80, SCREEN_HEIGHT // 2 - 6), 1)

def _draw_title(self, screen: pygame.Surface) -> None:
    """Renderiza el título y subtítulo del juego."""
    cx = SCREEN_WIDTH // 2

    # Título principal
    title_font = AssetLoader.get_font(None, 72)
    title_surface = title_font.render("El Último Bastión", True, Colors.GOLD)
    title_rect = title_surface.get_rect(center=(cx, SCREEN_HEIGHT // 2 - 110))
    screen.blit(title_surface, title_rect)

    # Sombra del título (desplazada 2px)
    shadow_surface = title_font.render("El Último Bastión", True, (20, 10, 0))
    shadow_rect = shadow_surface.get_rect(center=(cx + 2, SCREEN_HEIGHT // 2 - 108))
    screen.blit(shadow_surface, shadow_rect)
    screen.blit(title_surface, title_rect)

    # Subtítulo
    sub_font = AssetLoader.get_font(None, 22)
    sub_surface = sub_font.render(
        "El reino de Eldoria necesita un último defensor", True, Colors.LIGHT_GRAY
    )
    sub_rect = sub_surface.get_rect(center=(cx, SCREEN_HEIGHT // 2 - 40))
    screen.blit(sub_surface, sub_rect)

    # -----
    # Callbacks de botones
    # -----

def _on_play(self) -> None:
    self.state_machine.change(States.GAMEPLAY)

def _on_quit(self) -> None:
    self.event_bus.publish("quit_game")

"""
src/screens/gameplay.py
=====
Pantalla principal de juego.

```

En esta primera iteración (núcleo del motor) muestra:

- El mapa de fondo con su grid.
- HUD básico con marcadores de recursos y vida del bastión.

- Tecla ESC para pausar.

Los sistemas de oleadas, enemigos y torres se integrarán en iteraciones posteriores siguiendo la arquitectura propuesta.

"""

```
from __future__ import annotations
```

```
import pygame
```

```
from src.screens.base_screen import BaseScreen
```

```
from src.core.settings import SCREEN_WIDTH, SCREEN_HEIGHT, Colors, States
```

```
from src.core.asset_loader import AssetLoader
```

```
# Configuración del grid del mapa
```

```
GRID_COLS = 32
```

```
GRID_ROWS = 18
```

```
CELL_SIZE = 40      # píxeles por celda (1280 / 32 = 40)
```

```
MAP_OFFSET_X = 0
```

```
MAP_OFFSET_Y = 0
```

```
class GameplayScreen(BaseScreen):
```

```
    """
```

```
    Pantalla de juego activo.
```

```
    Attributes:
```

```
    _paused: Indica si el juego está pausado (flag interno).
```

```
    """
```

```
    def on_enter(self) -> None:
```

```
        """Inicializa el estado de la partida al entrar a gameplay."""
```

```
        self._wave_label = "Oleada: 0"
```

```
        self._gold = 200
```

```
        self._mana = 50
```

```
        self._bastion_hp = 100
```

```
        self._max_hp = 100
```

```
    def on_exit(self) -> None:
```

```
        pass
```

```
    # -----
```

```
    # Game loop
```

```
    # -----
```

```
    def handle_events(self, events: list[pygame.event.Event]) -> None:
```

```
        for event in events:
```



```

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE:
            self.state_machine.change(States.PAUSE)

```

```

def update(self, dt: float) -> None:

```

```

    """

```

```

    En esta iteración no hay lógica de juego activa todavía.
    Los sistemas se integrarán aquí en el siguiente sprint.

```

```

    """

```

```

def draw(self, screen: pygame.Surface) -> None:

```

```

    self._draw_map(screen)
    self._draw_hud(screen)
    self._draw_instructions(screen)

```

```

# -----
# Renderizado del mapa
# -----

```

```

def _draw_map(self, screen: pygame.Surface) -> None:

```

```

    """Dibuja el grid del mapa con alternancia de color de celdas."""

```

```

    color_a = (35, 55, 35) # Verde oscuro — terreno transitable
    color_b = (30, 48, 30) # Verde aún más oscuro — alternancia
    path_color = (90, 70, 40) # Marrón — camino de los enemigos

```

```

# Columnas del camino central (placeholder visual)
path_cols = set(range(0, GRID_COLS))
path_row = GRID_ROWS // 2

```

```

for row in range(GRID_ROWS):
    for col in range(GRID_COLS):
        x = MAP_OFFSET_X + col * CELL_SIZE
        y = MAP_OFFSET_Y + row * CELL_SIZE
        rect = pygame.Rect(x, y, CELL_SIZE, CELL_SIZE)

        # Dibujar celda del camino o terreno normal
        if row == path_row and col in path_cols:
            pygame.draw.rect(screen, path_color, rect)
        else:
            cell_color = color_a if (row + col) % 2 == 0 else color_b
            pygame.draw.rect(screen, cell_color, rect)

```

```

# Líneas del grid (muy sutiles)
pygame.draw.rect(screen, (20, 35, 20), rect, 1)

```

```

# Bastión al final del camino (margen derecho)
self._draw_bastion_placeholder(screen, path_row)

```

```

def _draw_bastion_placeholder(self, screen: pygame.Surface, path_row: int) -> None:
    """Dibuja un placeholder visual del bastión."""
    bx = MAP_OFFSET_X + (GRID_COLS - 2) * CELL_SIZE
    by = MAP_OFFSET_Y + (path_row - 1) * CELL_SIZE
    bw = CELL_SIZE * 2
    bh = CELL_SIZE * 3

    pygame.draw.rect(screen, (80, 70, 90), pygame.Rect(bx, by, bw, bh), border_radius=4)
    pygame.draw.rect(screen, Colors.LIGHT_GRAY, pygame.Rect(bx, by, bw, bh),
                     width=2, border_radius=4)

    font = AssetLoader.get_font(None, 13)
    label = font.render("BASTIÓN", True, Colors.GOLD)
    label_rect = label.get_rect(center=(bx + bw // 2, by + bh // 2))
    screen.blit(label, label_rect)

# -----
# HUD
# -----

def _draw_hud(self, screen: pygame.Surface) -> None:
    """Dibuja la barra HUD en la parte inferior de la pantalla."""
    hud_height = 60
    hud_rect = pygame.Rect(0, SCREEN_HEIGHT - hud_height, SCREEN_WIDTH,
hud_height)

    # Fondo del HUD
    hud_surface = pygame.Surface((SCREEN_WIDTH, hud_height), pygame.SRCALPHA)
    hud_surface.fill((20, 15, 30, 210))
    screen.blit(hud_surface, (0, SCREEN_HEIGHT - hud_height))

    # Borde superior del HUD
    pygame.draw.line(screen, Colors.GOLD,
                     (0, SCREEN_HEIGHT - hud_height),
                     (SCREEN_WIDTH, SCREEN_HEIGHT - hud_height), 2)

    font = AssetLoader.get_font(None, 22)
    y_center = SCREEN_HEIGHT - hud_height // 2

    # Vida del bastión
    hp_text = f"♥ Bastión: {self._bastion_hp} / {self._max_hp}"
    hp_surf = font.render(hp_text, True, Colors.BLOOD_RED)
    screen.blit(hp_surf, hp_surf.get_rect(midleft=(20, y_center)))

    # Oro
    gold_text = f"♦ Oro: {self._gold}"
    gold_surf = font.render(gold_text, True, Colors.GOLD)

```

```

        screen.blit(gold_surf, gold_surf.get_rect(center=(SCREEN_WIDTH // 2 - 140,
y_center)))

    # Maná
    mana_text = f"♦ Maná: {self._mana}"
    mana_surf = font.render(mana_text, True, Colors.MANA_BLUE)
    screen.blit(mana_surf, mana_surf.get_rect(center=(SCREEN_WIDTH // 2 + 60,
y_center)))

    # Oleada
    wave_surf = font.render(self._wave_label, True, Colors.PARCHMENT)
    screen.blit(wave_surf, wave_surf.get_rect(midright=(SCREEN_WIDTH - 20, y_center)))

def _draw_instructions(self, screen: pygame.Surface) -> None:
    """Muestra un recordatorio de controles en la esquina superior izquierda."""
    font = AssetLoader.get_font(None, 16)
    hint = font.render("ESC — Pausar", True, Colors.GRAY)
    screen.blit(hint, (12, 10))

```

"""

src/screens/pause_screen.py
=====

Pantalla de pausa.

Se muestra sobre el estado de gameplay (sin destruirlo).
Ofrece las opciones: Reanudar y Salir al menú.

"""

```
from __future__ import annotations
```

```
import pygame
```

```

from src.screens.base_screen import BaseScreen
from src.core.settings import SCREEN_WIDTH, SCREEN_HEIGHT, Colors, States
from src.core.asset_loader import AssetLoader
from src.ui.button import Button

```

```
class PauseScreen(BaseScreen):
```

"""

Pantalla de pausa superpuesta al juego.

Renderiza un overlay semitransparente con un panel central
y los botones de acción.

"""

```
def on_enter(self) -> None:
```

```

cx = SCREEN_WIDTH // 2
btn_w, btn_h = 260, 52
gap = 16
start_y = SCREEN_HEIGHT // 2 + 20

self._buttons = [
    Button(
        x=cx - btn_w // 2, y=start_y,
        width=btn_w, height=btn_h,
        label="Reanudar",
        callback=self._on_resume,
    ),
    Button(
        x=cx - btn_w // 2, y=start_y + btn_h + gap,
        width=btn_w, height=btn_h,
        label="Menú Principal",
        callback=self._on_main_menu,
    ),
]

# Surface de overlay (se crea una vez al entrar)
self._overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT),
pygame.SRCALPHA)
self._overlay.fill((0, 0, 0, 150))

def on_exit(self) -> None:
    self._buttons = []

# -----
# Game loop
# -----

def handle_events(self, events: list[pygame.event.Event]) -> None:
    for event in events:
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            self._on_resume()
        for button in self._buttons:
            button.handle_event(event)

def update(self, dt: float) -> None:
    pass

def draw(self, screen: pygame.Surface) -> None:
    # Overlay oscuro sobre el gameplay
    screen.blit(self._overlay, (0, 0))

    # Panel central
    self._draw_panel(screen)

```

```

        for button in self._buttons:
            button.draw(screen)

# -----
# Renderizado
# -----

def _draw_panel(self, screen: pygame.Surface) -> None:
    panel_w, panel_h = 360, 260
    px = SCREEN_WIDTH // 2 - panel_w // 2
    py = SCREEN_HEIGHT // 2 - panel_h // 2

    pygame.draw.rect(screen, (35, 25, 50),
                      pygame.Rect(px, py, panel_w, panel_h), border_radius=10)
    pygame.draw.rect(screen, Colors.GOLD,
                      pygame.Rect(px, py, panel_w, panel_h),
                      width=2, border_radius=10)

    font = AssetLoader.get_font(None, 42)
    title = font.render("PAUSA", True, Colors.PARCHMENT)
    title_rect = title.get_rect(center=(SCREEN_WIDTH // 2, py + 60))
    screen.blit(title, title_rect)

# -----
# Callbacks
# -----

def _on_resume(self) -> None:
    self.state_machine.change(States.GAMEPLAY)

def _on_main_menu(self) -> None:
    self.state_machine.change(States.MAIN_MENU)

"""
src/screens/defeat_screen.py
=====
Pantalla de derrota (Game Over).

Se muestra cuando la vida del bastión llega a cero.
Ofrece la opción de volver al menú principal.
"""

from __future__ import annotations

import pygame

```

```
from src.screens.base_screen import BaseScreen
from src.core.settings import SCREEN_WIDTH, SCREEN_HEIGHT, Colors, States
from src.core.asset_loader import AssetLoader
from src.ui.button import Button
```

```
class DefeatScreen(BaseScreen):
```

```
    """
```

```
    Pantalla de derrota.
```

```
    Muestra el mensaje "Game Over" y el botón para
    volver al menú principal.
```

```
    """
```

```
    def on_enter(self) -> None:
```

```
        cx = SCREEN_WIDTH // 2
```

```
        btn_w, btn_h = 280, 52
```

```
        self._button = Button(
```

```
            x=cx - btn_w // 2,
```

```
            y=SCREEN_HEIGHT // 2 + 80,
```

```
            width=btn_w,
```

```
            height=btn_h,
```

```
            label="Volver al Menú",
```

```
            callback=self._on_main_menu,
```

```
        )
```

```
    def on_exit(self) -> None:
```

```
        pass
```

```
    # -----
```

```
    # Game loop
```

```
    # -----
```

```
    def handle_events(self, events: list[pygame.event.Event]) -> None:
```

```
        for event in events:
```

```
            self._button.handle_event(event)
```

```
    def update(self, dt: float) -> None:
```

```
        pass
```

```
    def draw(self, screen: pygame.Surface) -> None:
```

```
        # Fondo muy oscuro
```

```
        overlay = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT),
        pygame.SRCALPHA)
```

```
        overlay.fill((10, 0, 0, 220))
```

```
        screen.blit(overlay, (0, 0))
```

```

cx = SCREEN_WIDTH // 2
cy = SCREEN_HEIGHT // 2

# "GAME OVER"
title_font = AssetLoader.get_font(None, 86)
title_surf = title_font.render("GAME OVER", True, Colors.BLOOD_RED)
screen.blit(title_surf, title_surf.get_rect(center=(cx, cy - 60)))

# Mensaje temático
sub_font = AssetLoader.get_font(None, 24)
sub_surf = sub_font.render(
    "El Último Bastión ha caído. Eldoria está perdida.", True, Colors.LIGHT_GRAY
)
screen.blit(sub_surf, sub_surf.get_rect(center=(cx, cy + 10)))

self._button.draw(screen)

# -----
# Callbacks
# -----

def _on_main_menu(self) -> None:
    self.state_machine.change(States.MAIN_MENU)

```

"""

src/ui/button.py

=====

Componente botón reutilizable para la interfaz de usuario.

Soporta estado normal, hover y presionado. Llama a un callback cuando el jugador hace clic con el botón izquierdo del ratón.

"""

```

from __future__ import annotations
from typing import Callable

```

```

import pygame

```

```

from src.core.settings import Colors
from src.core.asset_loader import AssetLoader

```

```

class Button:

```

"""

Botón interactivo de la UI.

Attributes:

rect: Área clickeable y de renderizado.

label: Texto que muestra el botón.
callback: Función a ejecutar al hacer clic.
is_hovered: True cuando el cursor está sobre el botón.

"""

Colores por estado

COLOR_NORMAL = (60, 45, 80)

COLOR_HOVER = (90, 65, 120)

COLOR_PRESSED = (40, 30, 55)

COLOR_BORDER = (180, 140, 60) # Dorado medieval

COLOR_TEXT = Colors.PARCHMENT

BORDER_WIDTH = 2

BORDER_RADIUS = 6

def __init__(

self,

x: int,

y: int,

width: int,

height: int,

label: str,

callback: Callable,

font_size: int = 24,

) -> None:

self.rect = pygame.Rect(x, y, width, height)

self.label = label

self.callback = callback

self.font_size = font_size

self.is_hovered: bool = False

self._is_pressed: bool = False

Actualización

def handle_event(self, event: pygame.event.Event) -> None:

"""

Procesa un evento individual de Pygame.

Debe llamarse desde la pantalla que contiene este botón.

"""

if event.type == pygame.MOUSEMOTION:

self.is_hovered = self.rect.collidepoint(event.pos)

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:

if self.rect.collidepoint(event.pos):

self._is_pressed = True


```

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self._is_pressed and self.rect.collidepoint(event.pos):
                self.callback()
                self._is_pressed = False

# -----
# Renderizado
# -----

def draw(self, screen: pygame.Surface) -> None:
    """Dibuja el botón en la surface proporcionada."""
    # Seleccionar color de fondo según estado
    if self._is_pressed:
        bg_color = self.COLOR_PRESSED
    elif self.is_hovered:
        bg_color = self.COLOR_HOVER
    else:
        bg_color = self.COLOR_NORMAL

    # Fondo con bordes redondeados
    pygame.draw.rect(screen, bg_color, self.rect, border_radius=self.BORDER_RADIUS)

    # Borde dorado
    pygame.draw.rect(
        screen, self.COLOR_BORDER, self.rect,
        width=self.BORDER_WIDTH, border_radius=self.BORDER_RADIUS
    )

    # Texto centrado
    font = AssetLoader.get_font(None, self.font_size)
    text_surface = font.render(self.label, True, self.COLOR_TEXT)
    text_rect = text_surface.get_rect(center=self.rect.center)
    screen.blit(text_surface, text_rect)

"""
tests/test_event_bus.py
=====
Pruebas unitarias para el EventBus.
No requieren Pygame para ejecutarse.
"""

import unittest
from src.core.event_bus import EventBus

class TestEventBus(unittest.TestCase):

```

```

def setUp(self):
    EventBus.reset()
    self.bus = EventBus.get_instance()

def test_singleton(self):
    """Dos llamadas a get_instance devuelven el mismo objeto."""
    bus2 = EventBus.get_instance()
    self.assertIs(self.bus, bus2)

def test_subscribe_and_publish(self):
    """Un suscriptor recibe los datos al publicar el evento."""
    received = []
    self.bus.subscribe("test_event", lambda data: received.append(data))
    self.bus.publish("test_event", {"value": 42})
    self.assertEqual(received, [{"value": 42}])

def test_multiple_subscribers(self):
    """Múltiples suscriptores reciben el mismo evento."""
    log = []
    self.bus.subscribe("ev", lambda d: log.append("a"))
    self.bus.subscribe("ev", lambda d: log.append("b"))
    self.bus.publish("ev")
    self.assertIn("a", log)
    self.assertIn("b", log)

def test_unsubscribe(self):
    """Un suscriptor desuscrito no recibe más eventos."""
    log = []
    cb = lambda d: log.append(d)
    self.bus.subscribe("ev", cb)
    self.bus.unsubscribe("ev", cb)
    self.bus.publish("ev", "data")
    self.assertEqual(log, [])

def test_no_duplicate_subscriptions(self):
    """El mismo callback no se registra dos veces."""
    log = []
    cb = lambda d: log.append(d)
    self.bus.subscribe("ev", cb)
    self.bus.subscribe("ev", cb)
    self.bus.publish("ev", 1)
    self.assertEqual(len(log), 1)

def test_publish_without_subscribers(self):
    """Publicar a un evento sin suscriptores no lanza excepción."""
    try:
        self.bus.publish("orphan_event", {"data": 1})
    except Exception as e:

```

```
self.fail(f"publish lanzó excepción inesperada: {e}")
```

```
def test_clear(self):  
    """clear() elimina todas las suscripciones."""  
    log = []  
    self.bus.subscribe("ev", lambda d: log.append(d))  
    self.bus.clear()  
    self.bus.publish("ev", "after_clear")  
    self.assertEqual(log, [])
```

```
if __name__ == "__main__":  
    unittest.main()
```